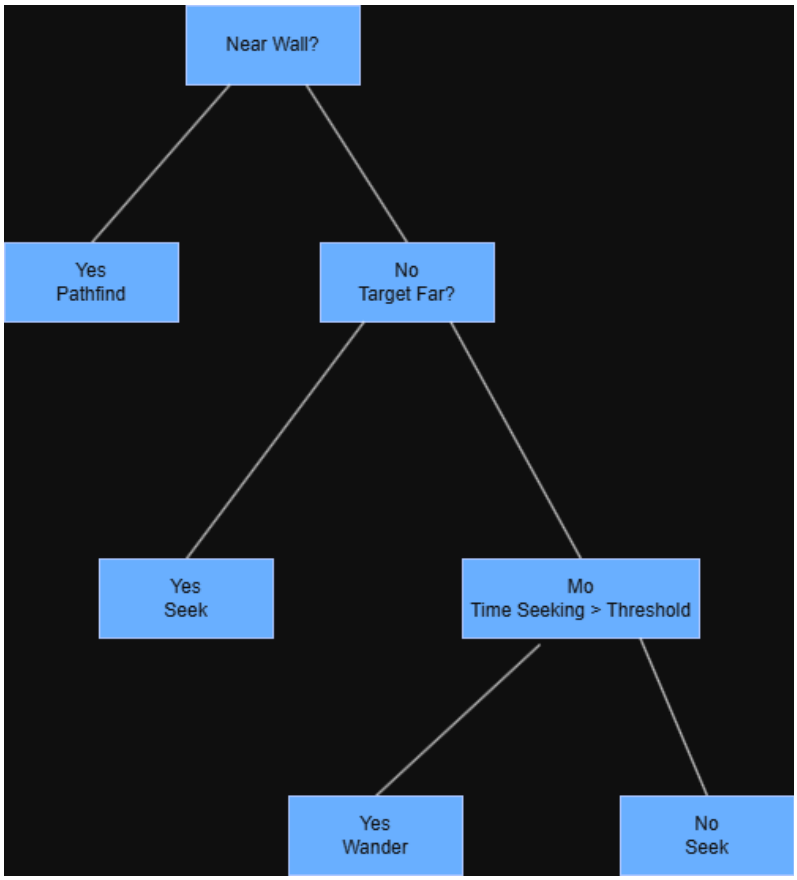
Decision-Making Process Letter

The Process of completing this assignment started by building off and refactoring my code from the pathfinding assignment. The main components I needed were A* search and my NavGrid environment. The Nav environment and scenes were updated to include descriptions and various tracking metrics, as well as an updated build mode that pauses the simulation. After that, it was to tackle each algorithm, the **Decision Tree**, **Behavior Tree**, and **GOAP Planner**, in order, starting with the decision tree. The first task was to come up with the '**task**' that my AI agent would utilize and keep it consistent throughout all the algorithms. Another organizational step I could have taken was to make a shared file that houses that structure so that I wasn't referencing the decision tree header. The final design I landed on was to create 3 scenes that are toggleable at runtime, which will build and execute each tree/planner. At first, my ofApp housed all the logic, but I made separate scripts for a cleaner and more readable project.

Decision Tree

The Decision Tree controls a single agent that switches between movement modes such as **dormant**, **seek, wander, and pathfinding** based on environmental conditions like distance to a target, time spent seeking, and proximity to walls. The tree is structured as a layered set of condition checks (3+ levels deep), where each branch evaluates context values stored in a shared DecisionContext. The agent continuously evaluates the tree each frame and updates its movement mode accordingly.




```
void ofApp::buildDecisionTree() {
    auto dormantAction = std::make_unique<ActionNode>(
        [](DecisionContext& ctx) {
            ctx.chosenMode = MovementMode::DORMANT;
        });

    auto pathfollowAction = std::make_unique<ActionNode>(
        [](DecisionContext& ctx) {
            ctx.chosenMode = MovementMode::PATHFOLLOW;
        });

    auto seekAction = std::make_unique<ActionNode>(
        [](DecisionContext& ctx) {
            ctx.chosenMode = MovementMode::SEEK;
        });

    auto wanderAction = std::make_unique<ActionNode>(
        [](DecisionContext& ctx) {
            ctx.chosenMode = MovementMode::WANDER;
        });

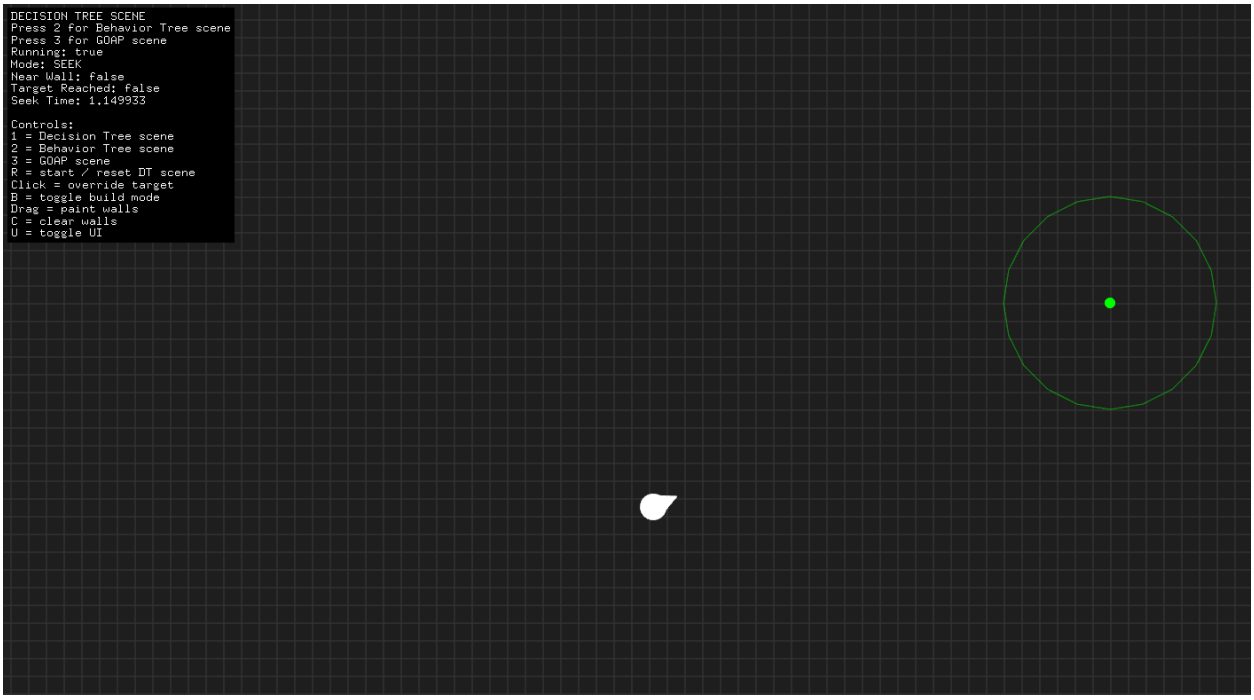
    auto targetDistanceQuestion = std::make_unique<QuestionNode>(
        [](const DecisionContext& ctx) {
            return ctx.targetFarEnough;
        },
        std::move(seekAction),
        std::move(wanderAction));

    auto seekTimeQuestion = std::make_unique<QuestionNode>(
        [this](const DecisionContext& ctx) {
            return ctx.currentMode == MovementMode::SEEK &&
                ctx.timeSeeking >= seekTimeThreshold_;
        },
        std::make_unique<ActionNode>(
            [](DecisionContext& ctx) {
                ctx.chosenMode = MovementMode::WANDER;
            }),
        std::move(targetDistanceQuestion));

    auto nearWallQuestion = std::make_unique<QuestionNode>(
        [](const DecisionContext& ctx) {
            return ctx.nearWall;
        },
        std::move(pathfollowAction),
        std::move(seekTimeQuestion));

    decisionTree_ = std::make_unique<QuestionNode>(
        [](const DecisionContext& ctx) {
            return !ctx.isRunning;
        },
        std::move(dormantAction),
        std::move(nearWallQuestion));
}
```

One important improvement I made was ensuring that pathfinding persists until the target has made it away from the walls and has a clear path to the target, instead of switching states too early. This helped fix issues where the agent would get stuck or oscillate between behaviors. Due to the nature of the environment, when the target changes (which it does on set intervals), the pathfinding will take a few seconds to update.



Behavior Tree

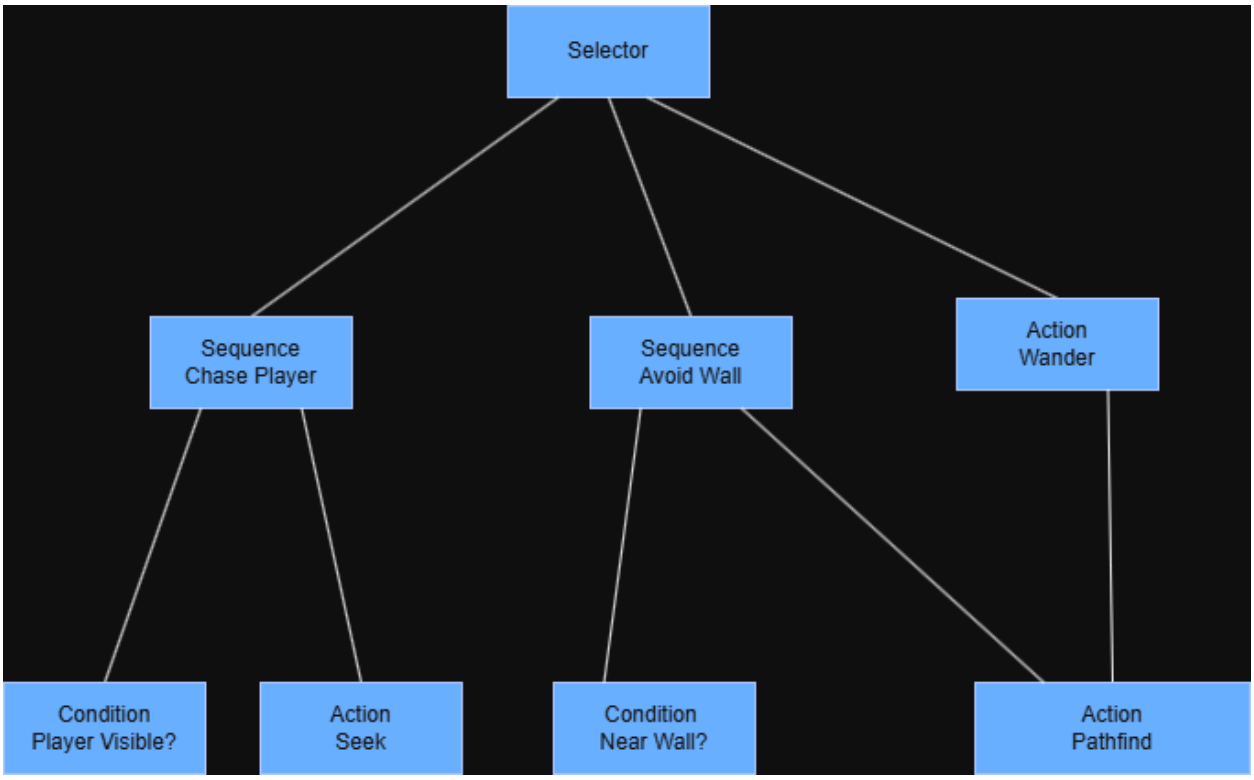
The Behavior Tree controls both a player agent and a monster agent, using a modular node system with different task types such as **Sequence, Selector, and condition/action nodes**.

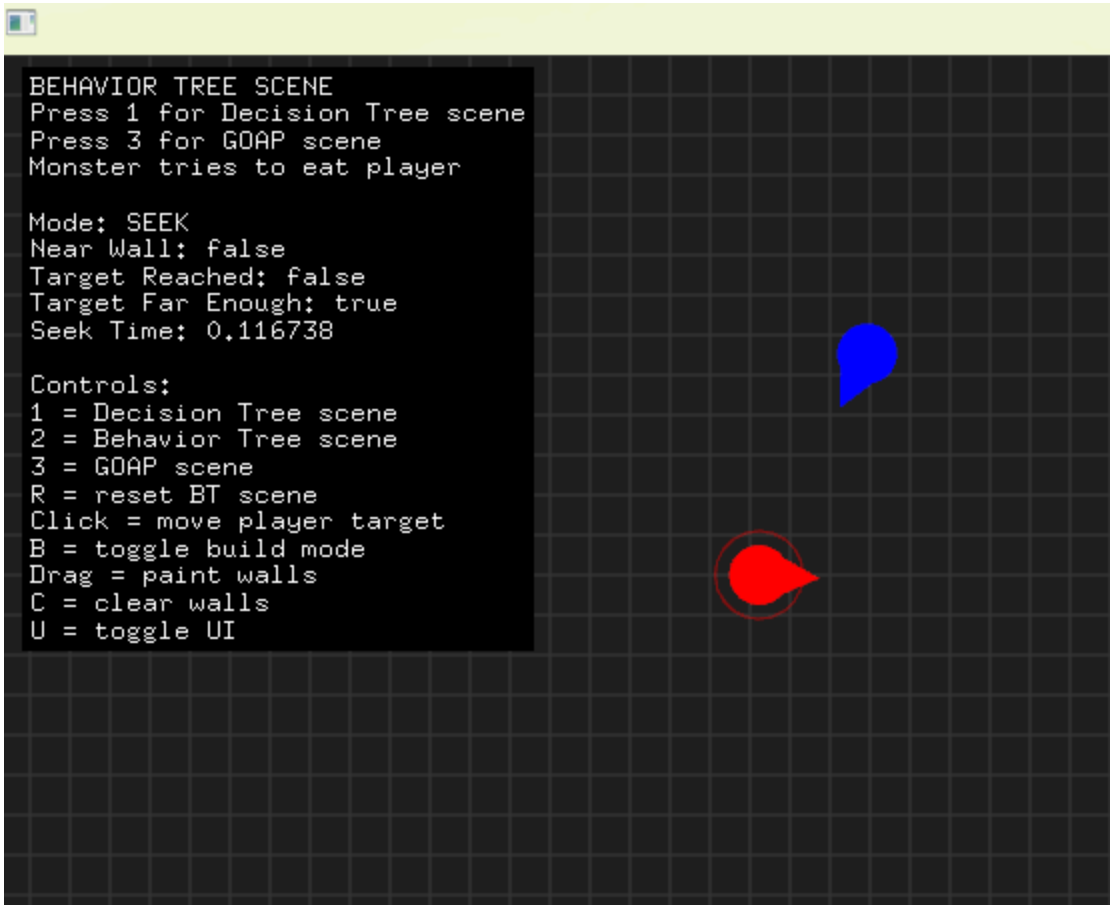
The monster uses the Behavior Tree to decide between behaviors like:

- Seeking the player
- Wandering when no target is available
- Pathfinding when blocked

The system uses a shared BehaviorContext to pass data such as position, velocity, and environmental checks into the tree each frame. This allows the tree to remain modular and reusable across scenes, including the GOAP monster.

Compared to the Decision Tree, this approach made it much easier to extend behavior without rewriting logic, since new nodes can be added without restructuring the entire system.


Selector
Sequence
Chase Player
Sequence
Avoid Wall
Action
Wander
Condition
Player Visible?
Action
Seek
Condition
Near Wall?
Action
Pathfind


BEHAVIOR TREE SCENE
Press 1 for Decision Tree scene
Press 3 for GOAP scene
Monster tries to eat player
Mode: SEEK
Near Wall: false
Target Reached: false
Target Far Enough: true
Seek Time: 0.116738
Controls:
1 = Decision Tree scene
2 = Behavior Tree scene
3 = GOAP scene
R = reset BT scene
Click = move player target
B = toggle build mode
Drag = paint walls
C = clear walls
U = toggle UI

```cpp
//--------------------------------------------------------------
void ofApp::buildBehaviorTree() {
    behaviorTreeStorage_.clear();

    auto makeNode = [this](auto* node) -> auto* {
        behaviorTreeStorage_.emplace_back(node);
        return node;
        };

    auto root = makeNode(new bt::Selector(&btContext_));

    auto eatBranch = makeNode(new bt::Sequence(&btContext_));
    eatBranch->addChild(makeNode(new bt::TargetReachedCondition(&btContext_)));
    eatBranch->addChild(makeNode(new bt::SetModeAction(&btContext_, MovementMode::DORMANT)));

    auto wallBranch = makeNode(new bt::Sequence(&btContext_));
    wallBranch->addChild(makeNode(new bt::NearWallCondition(&btContext_)));
    wallBranch->addChild(makeNode(new bt::SetModeAction(&btContext_, MovementMode::PATHFOLLOW)));

    auto timeoutBranch = makeNode(new bt::Sequence(&btContext_));
    timeoutBranch->addChild(makeNode(new bt::SeekTimedOutCondition(&btContext_, seekTimeThreshold_)));
    timeoutBranch->addChild(makeNode(new bt::SetModeAction(&btContext_, MovementMode::WANDER)));

    auto seekBranch = makeNode(new bt::Sequence(&btContext_));
    seekBranch->addChild(makeNode(new bt::TargetFarEnoughCondition(&btContext_)));
    seekBranch->addChild(makeNode(new bt::SetModeAction(&btContext_, MovementMode::SEEK)));

    auto fallback = makeNode(new bt::SetModeAction(&btContext_, MovementMode::WANDER));

    root->addChild(eatBranch);
    root->addChild(wallBranch);
    root->addChild(timeoutBranch);
    root->addChild(seekBranch);
    root->addChild(fallback);

    behaviorTreeRoot_ = root;
```

GOAP Planner

The GOAP system controls the player agent, while the monster continues using a Behavior Tree. This demonstrates how different AI systems can interact in the same environment.

The planner uses a set of actions with:

- Preconditions
- Effects
- Costs

Examples from my implementation include:

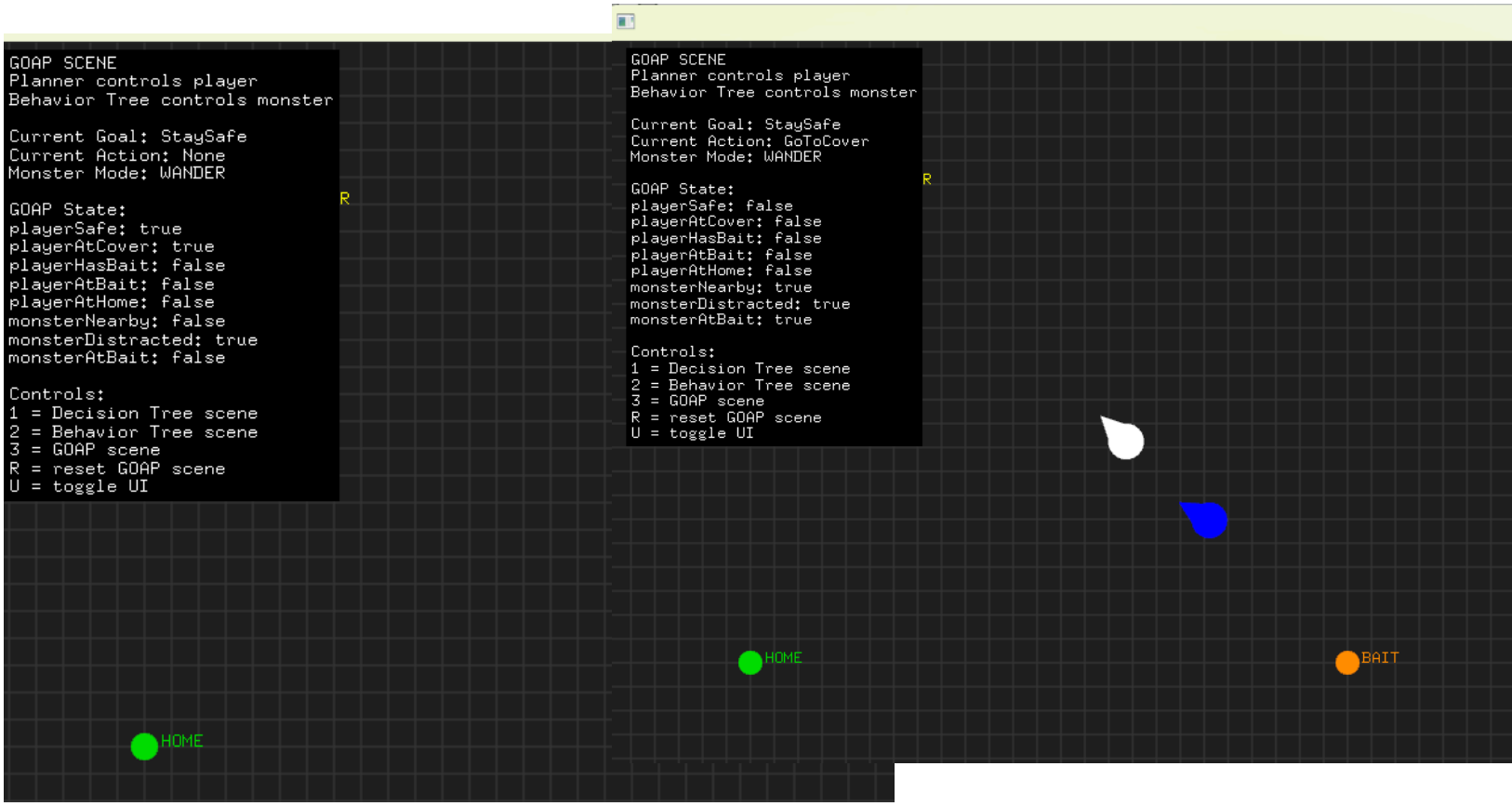
- *GoToBait*
- *PickUpBait*
- *DropBait*
- *GoHome*

At each frame, the system updates the world state, selects a goal such as **StaySafe** or **DistractMonster**, and then generates a plan dynamically. The player then executes actions sequentially, updating the world state and triggering replanning when needed. One key feature I added was dynamic replanning based on timers and state changes, which allows the agent to react if the monster moves or the situation changes.

All three AI systems share core functionality through ofApp, including:

- NavGrid + A* for pathfinding
- Boid movement system for motion
- Obstacle-building mode for testing environments (Except for GOAP)
- Scene switching between Decision Tree, Behavior Tree, and GOAP

The function requestPathTo() converts grid paths into world-space waypoints for agents to follow. This shared structure allowed me to directly compare how each decision-making system behaves under the same conditions.



What I learned is that each decision-making algorithm has its own strengths based on complexity and the task you want. I think Behavior Trees are best for enemies with complex behaviors and for combat. GOAP is good for goal-driven systems, and decision trees are for simple behaviors.